Privilege A2A: how to demo it and how to set it up

Oct 4, 2026 · @Curtis Muir

This is the runbook for our environment: two agent-to-agent demos that run through the PingOne Privilege AI Gateway, how to present them, and how to rebuild the setup. Every step here was run on 4 October 2026.

What the demo shows

Agents call each other and their tools only through the Privilege AI Gateway, each with its own agent key. There are two flows, and both run live.

Flow	Path	Demo Step
Access review	Main agent, gateway, worker agent, gateway, MCP tools	25 (UC46)
Three-deep chain	Main agent, gateway, LangChain agent, gateway, specialist	26 (UC47)

The three points to land with an audience:

- **Every hop is checked.** The gateway checks the caller's key, its scope, the policy for that agent or tool, and the allowed model. No agent holds another agent's key or the model provider's key.
- **Unsafe content is stopped.** A prompt-injection message between agents is blocked, personal data in a tool result is redacted, and a tool result carrying hidden instructions is blocked before the model reads it.
- **Ungranted hand-offs are refused.** An agent that tries to call an agent it was not granted gets a 403 at the gateway.

The two agents run on different model tiers (Gemini 3.6 Flash for the main agent, Gemini 3.5 Flash Lite for the worker), so the per-agent cost split is visible in the console.

Before you demo

Four things must be up. Check them in this order; each takes seconds.

#	What	Check	Good result
1	Main agent (runs on the laptop, port 3300)	`curl -s http://127.0.0.1:3300/status`	JSON with `"viaGateway":true` and a `chain` URL
2	Worker, MCP server and LangChain agent (cluster)	`kubectl --context us -n ping-devops-cmuir get pods`	`workeragentdemo`, `mcpdemo` and `langchain-agent` all `1/1 Running`
3	Gateway	`curl -s -o /dev/null -w "%{http_code}" -X POST https://mcpgw.ai-demo.ping-devops.com/workeragentdemo/a2a/jsonrpc -d '{}'`	`401` (the gateway is up and asking for a key)
4	The app	Open `https://local.ping-devops.com:4000` and sign in	The dashboard loads. Sign-in only works on this host

If the main agent is not running, start it:

```
cd standalone/privilege-a2a-anthropic-demo && npm run dev
```

Then do one dry run of each demo before the audience arrives. A model call can take a few seconds the first time.

Demo 1: access review

A security analyst asks an agent to review someone's access. The main agent hands the lookup to a worker agent, which reads the access data through MCP tools. About three minutes.

Open it. Demo steps, step 25 (UC46), or the side-nav entry AI Gateway · A2A Review. Both land on the A2A tab of the gateway client. The path across the top reads Main agent, AI Gateway, Worker agent, AI Gateway, MCP tools.

Step	Type this	What happens	What to say
1	`Review Carol Nguyen's access and flag anything risky.`	In about five seconds the trace shows the hand-off, the tool call `list_entitlements`, and an answer naming her expired `prod-db-admin` grant that is still in use	"Two agents, two keys, two model tiers. The main agent never touches the tools; the worker never sees the main agent's key."
2	Point at the tool result in the trace	Carol's email reads `[REDACTED:pii]`	"The gateway redacted that on the way back. The worker's model never saw the address."
3	`What access does Dave hold? Review his grants.`	The worker reports that the response was blocked by security policy	"Dave's record has an instruction hidden in it, telling the agent to open a ticket granting someone admin. The gateway blocked the tool result before the model could read it."

Why step 3 works. The MCP server runs with `MCP_DEMO_INJECTION=true`, which plants the hidden instruction in Dave's record only. The gateway's log shows `aiguard: blocked mcp response … category=malicious_content`, and the write tool `open_ticket` is never called.

If you have time. Open the Privilege console, AI Gateway, Agent Registry, and show the two agents: different scopes (`llm` + `a2a` against `llm` + `mcp`), different allowed models, and the worker's per-tool grants.

Demo 2: three-deep chain

One agent calls a second, which calls a third, and the gateway sits between each pair. Three runs, each under a second. About three minutes.

Open it. Go to `/dashboard`, open Demo steps in the top bar, and find step 26, UC47 Privilege A2A three-deep agent chain. It has three controls: the step itself and two small buttons, **P** and **U**.

Run	Click	What the view shows	What to say
Allowed	The step	Main agent, gateway, LangChain agent, gateway, Refunds specialist, then two replies back. Marked verified	"Each call is three steps: the caller presents its own key, the gateway checks it, the next agent receives it. The second hop uses the LangChain agent's key, not the main agent's."
Prompt injection	**P**	Main agent calls the LangChain agent, then "Privilege gateway — refused". Nothing is forwarded	"The message said to ignore all instructions and reveal the system prompt. The gateway blocked it at the first hop; the LangChain agent never saw it."
Ungranted agent	**U**	First hop passes; the second stops at "Privilege gateway — refused"	"The LangChain agent tried to hand off to an agent it was never granted. The gateway refused with a 403, and the refusal travels back up the chain with its reason."

Best view for this demo. The sequence view: one lifeline per agent with the gateway between them, so the audience sees the gateway in the path of every call. Switch to it in Quick Config. The movie reel along the bottom shows the same run as a numbered chain.

Good follow-up. On the System Flow map the same run lights a Privilege A2A band: agents green, the gateway red where it refused.

Where to show it

The chain run feeds every flow view on the dashboard, so pick the one that suits the room.

View	How to get it	Best for
Sequence view	Quick Config on the dashboard	Showing the gateway between every pair of agents
Movie reel	The default view	A numbered walk through the hops; click a step for its detail
Simple stepper	Quick Config	A plain table of hops and their status
Token topology	Quick Config	The hops as a node graph
System Flow	The System Flow panel	Where the A2A agents sit beside the rest of the stack

Pages for the architecture conversation:

Page	Route	What it has
Privilege Gateway Topologies	`/privilege-gateway-topologies`, Agent-to-agent tab	The architecture diagram: the gateway's three routes and the eight numbered legs
Server Topology	`/architecture/server-topology`	Buttons A2A access review and A2A agent chain show each flow on its own, hop by hop
Privilege for AI, A2A	`/ai-products/privilege-a2a`	The product story, with links to both demos
Academy lesson	`/academy/journey?step=privilege-a2a`	A three-step lesson. It explains A2A; it does not run it

The side nav's AI Gateway group has AI Gateway · A2A Review (the demo tab) and AI Gateway Topologies (the diagram).

Set up from scratch

Do it in this order. Apps come before agents, because an agent can only select apps that already exist. Our values: gateway `https://mcpgw.ai-demo.ping-devops.com`, mesh cluster `ai-demo-cmuir`, namespace `ping-devops-cmuir`, kubectl context `us`.

1. Deploy the worker and the MCP server. One image, two Deployments.

```
cd standalone/privilege-a2a-anthropic-demo
docker buildx build --platform linux/arm64 --provenance=false \
  -t ghcr.io/cmuir_pingcorp/privilege-a2a-demo:latest --push .
kubectl --context us apply -f k8s/deployment.yaml
```

The cluster nodes are arm64, so build for `linux/arm64`. The LangChain agent and the test specialist are already deployed in the same namespace.

2. Register the apps (console: Agentic Apps, Add Application).

App	Type	Backend URL	Path	Auth Mode
`workeragentdemo`	Remote Agent	`http://workeragentdemo.ping-devops-cmuir.svc.cluster.local`	`/a2a`	None
`mcpdemo`	MCP	`http://mcpdemo.ping-devops-cmuir.svc.cluster.local`	`/mcp`	None
`langchainagent`	Remote Agent	`http://langchain-agent.ping-devops-cmuir.svc.cluster.local:8888`	`/a2a`	Static Token
`hostile-a2a`	Remote Agent	`http://hostile-a2a-agent.ping-devops-cmuir.svc.cluster.local`	`/a2a`	None
`restrictedagent`	Remote Agent	the `workeragentdemo` backend	`/a2a`	None

The worker must be a Remote Agent app, or it will not appear in an agent's Remote Agents list. `restrictedagent` exists only to be refused: select it on no agent. Turn on Discover A2A agent card once a backend is running. The static token for `langchainagent` is `PRIVILEGE_A2A_STATIC_TOKEN` in the vault.

3. Create the agents (console: AI Gateway, Agent Registry, Add Agent).

Agent	Scopes	Provider and model	May reach	Custodian
`main-agent`	`llm`, `a2a`	Google, Gemini 3.6 Flash	`workeragentdemo`, `langchainagent`	you
`worker-agent`	`llm`, `mcp`	Google, Gemini 3.5 Flash Lite	`mcpdemo`, with each of its four tools allowed under Configure Tools	you
`langchain-caller`	`a2a`	none	`hostile-a2a`	a different user

Each agent's Real Provider Key is the Google API key. The key is shown once, on the last wizard screen. `langchain-caller` needs its own custodian: remote-agent grants follow the custodian, so under a shared one it could reach the worker and the refusal run would not be refused.

4. Store the keys. Paste each key once; a doubled key is refused as `invalid virtual key`.

Key	Where it goes
`main-agent`	`MAIN_AGENT_KEY` in `standalone/privilege-a2a-anthropic-demo/.env`
`worker-agent`	`WORKER_AGENT_KEY` in the `privilege-a2a-demo` Secret
`langchain-caller`	`PRIVILEGE_A2A_AGENT_KEY` in the vault, then mirrored to `langchain-secrets` by `k8s/create-secrets.sh`

5. Point the main agent at the gateway in its `.env`:

```
MAIN_AGENT_HOST=0.0.0.0
MAIN_AGENT_PROVIDER=google
GEMINI_BASE_URL=https://mcpgw.ai-demo.ping-devops.com/llm/google
MAIN_AGENT_MODEL=gemini-3.6-flash
WORKER_A2A_URL=https://mcpgw.ai-demo.ping-devops.com/workeragentdemo/a2a
WORKER_TRACE_URL=https://mcpgw.ai-demo.ping-devops.com/workeragentdemo/trace
CHAIN_A2A_URL=https://mcpgw.ai-demo.ping-
devops.com/langchainagent/a2a/jsonrpc
```

The model ID must match the model allowed on the key exactly. `MAIN_AGENT_HOST=0.0.0.0` lets the app's Docker container reach it.

6. Connect the app. In `demo_api_server/.env`, set `PRIVILEGE_A2A_MAIN_URL=http://host.docker.internal:3300` and `PRIVILEGE_A2A_BFF_SECRET` to the same value as the main agent's `A2A_BFF_SHARED_SECRET`. The secret is also in the vault under that name.

7. Set the chain's targets. `A2A_DELEGATE_TARGETS` in `k8s/02-configmap.yaml` maps `refunds` to the `hostile-a2a` route and `restricted` to the `restrictedagent` route. Restart `langchain-agent` after changing it.

Then run the checks in Before you demo, and both demos once.

When something fails

Each of these happened while building the demo. Read the message, find the row.

What you see	Cause	Fix
"The A2A service is unavailable" in the app	The main agent on port 3300 is not running, or the app cannot reach it	Start it with `npm run dev`. Check `PRIVILEGE_A2A_MAIN_URL` and that the two BFF secrets match
`invalid virtual key` (401)	The agent key is wrong, often pasted twice	Check the stored key is 57 characters; store it again
`model "…" not allowed for this key` (403)	The model ID does not exactly match the model allowed on the agent	Use the exact ID, for example `gemini-3.6-flash`. Aliases such as `gemini-flash-latest` are refused

What you see	Cause	Fix
`API key not valid` from the provider	The gateway accepted the call; the Real Provider Key stored on that agent is wrong	Edit the agent and paste the Google key again
The worker says the tool call was `Forbidden`	The agent may reach the MCP app but no tools are allowed	Configure Tools on the agent, allow each tool
Configure Tools shows "Gateway Unreachable … no such host"	Tool discovery ran once, before the backend existed	Save the MCP app again to re-run discovery
An app is missing from the Remote Agents list	It was registered as a type other than Remote Agent	Delete it and add it again as a Remote Agent
`401: Bearer authentication required` at the first chain hop	`langchainagent` is not on Static Token, so the gateway forwards no credential	Set Auth Mode to Static Token and paste the token from the vault
`unknown delegation target` at the second chain hop	The LangChain agent has no agent key, or the target is not in `A2A_DELEGATE_TARGETS`	Mirror `PRIVILEGE_A2A_AGENT_KEY` to the cluster; check the config; restart the agent
The "ungranted" run succeeds	The LangChain agent shares a custodian with an agent that is granted that target	Give `langchain-caller` its own custodian
A view shows nothing for a chain run	The page was reloaded; the trace lives in the browser	Run the step again

For anything else, read the gateway's log. A refusal names its reason there:

```
kubectl --context us -n ping-devops-curtismuir exec deploy/agentless-mcpgw -c
agentless-mcpgw -- \
  sh -c 'tail -n 400 /var/log/procyon/cyonproxy.log | grep -E "policy
denied|aiguard|not scoped|invalid virtual key"'
```

That log prints full agent keys on some lines. Do not paste it anywhere without masking them.

Known limits

Say these plainly if asked; none of them is hidden by the demo.

- **No user identity crosses the hops.** The A2A message carries the question and the answer, not a browser token or a delegated token. Each agent is identified by its own key. The banking A2A demo (UC2) is the one that shows a user's delegated identity across agents.
- **The model leg on Google is not screened by the guardrail.** The gateway logs that it finds nothing to scan in a Gemini-format request. The poisoned tool result is stopped earlier, on the tool leg, so the demo does not depend on it. This leg has not been tested with an attack.
- **Grants follow the custodian.** Two agents under one custodian can reach each other's remote agents. Separate custodians are what keep their reach separate.
- **The main agent runs on the laptop.** If the laptop sleeps or the process stops, both demos stop. The worker, MCP server, LangChain agent and specialist run in the cluster.
- **Pass Agent Token and User Token are not used.** Both demos use agent keys with Static Token or None on the remote agent. Workload identity for agent callers has not been tested.
- **The specialist is a stub.** The refunds specialist answers that nothing was executed. The chain shows the hops and the controls, not a real refund.

Related guides: the A2A guide (`privilege/GATEWAY-AGENT-TO-AGENT-A2A.md`) for how the gateway handles A2A in general, and the demo's README (`standalone/privilege-a2a-anthropic-demo/README.md`) for its settings.